

With the recent development of powerful CPUs, is it still necessary to write highly efficient code?

Gaspar Sekula

CONTACT@GASPARSEKULA.COM

Abstract

The recent spread of high-performance, many-core processors has fostered the misconception that hardware abundance mitigates the need for software efficiency. In this study, I challenge this "hardware-first" paradigm by benchmarking naive, JIT-compiled (Numba), and library-optimized (NumPy) implementations of matrix multiplication on an HPC cluster. The experiments reveal a staggering $70,000\times$ performance gap on identical hardware, demonstrating that unoptimized code leaves most of modern CPU throughput inaccessible. While optimized solutions achieve super-linear speedups, naive implementations remain fundamentally incapable of scaling, regardless of core count. These findings conclude that powerful CPUs act as a latent resource that only highly efficient code can activate, making algorithmic optimization more critical in the era of Big Data and AI than ever before. The codebase is available in repository: <https://github.com/GasparSekula/CodeEfficiency.git>.

1. Introduction

The exponential growth of Big Data systems requires efficient processing of gigabytes or petabytes of data, where software efficiency is associated with scalability, energy consumption, and cost at a massive scale. While Moore's Law (Moore, 1965) has historically provided a consistent increase in transistor density, the *Software Paradox* — often summarized by Wirth's Law (Wirth, 1995) — suggests that software complexity frequently outpaces hardware gains. A better choice of code components impacts the organisation's long-term financial performance and CO2 emissions footprint (Strubell et al., 2019).

Matrix multiplication ($\mathcal{O}(n^3)$) serves as a critical proxy for this performance gap, particularly given its foundational role in training large-scale Artificial Neural Networks (ANNs). While modern hardware provides massive theoretical throughput, high-level languages like Python introduce significant abstractions, such as dynamic type checking and the Global Interpreter Lock (GIL), which can leave modern CPU resources largely underutilized. This paper examines whether modern many-core processors make efficient coding less important, or if they actually increase the need for tools like NumPy and Numba to maximize performance.

2. Methodology

While a naive implementation of matrix multiplication utilizes a triple-nested loop, high-performance alternatives such as NumPy and Numba leverage lower-level optimizations to accelerate computation. In the naive approach, the Python interpreter executes operations

sequentially; as a dynamic language, Python incurs significant temporal overhead due to runtime type checking and reference counting.

In contrast, NumPy (Harris et al., 2020) serves as a wrapper for highly optimized C and Fortran libraries (e.g., OpenBLAS or MKL). These libraries utilize tiling—partitioning matrices into blocks that fit within the CPU’s caches—to maximize data locality and minimize memory latency (Lam et al., 1991). Numba employs a Just-In-Time (JIT) compiler to translate Python bytecode into machine code via the LLVM compiler infrastructure (Lam et al., 2015). This allows for the bypass of the Global Interpreter Lock (GIL) and the parallel distribution of iterations across available physical CPU cores.

GFLOPS (GigaFLOPS) is a unit used to measure a computer’s processing power (Hennessy and Patterson, 2011), equivalent to one billion floating-point operations per second. In our analysis, the following formula is used:

$$\text{GFLOPS} = \frac{2 \cdot N^3}{10^9 \cdot t},$$

where N is the size of matrix and t is the multiplication time.

This study evaluates four distinct approaches: naive, NumPy, Numba, and a GPU-accelerated baseline using CuPy (Okuta et al., 2017). The primary metric of evaluation is execution latency. While numerical precision is vital in scientific computing, this study focuses on execution time as a proxy for computational efficiency, reflecting the cost-per-unit-time model prevalent in modern cloud-based High-Performance Computing (HPC).

Size	Naive	NumPy	Numba	CuPy
10	$0.0009_{\pm < 10^{-4}}$	$< \mathbf{10^{-4}}_{\pm < 10^{-4}}$	$< \mathbf{10^{-4}}_{\pm < 10^{-4}}$	$0.0066_{\pm 0.0204}$
50	$0.0677_{\pm 0.0204}$	$< \mathbf{10^{-4}}_{\pm < 10^{-4}}$	$< \mathbf{10^{-4}}_{\pm < 10^{-4}}$	$0.0001_{\pm < 10^{-4}}$
100	$0.4457_{\pm 0.0016}$	$< \mathbf{10^{-4}}_{\pm < 10^{-4}}$	$0.0001_{\pm < 10^{-4}}$	$0.0002_{\pm < 10^{-4}}$
250	$6.6268_{\pm 0.2527}$	$0.0006_{\pm < 10^{-4}}$	$0.0014_{\pm < 10^{-4}}$	$0.0004_{\pm 0.0006}$
500	$52.3054_{\pm 0.3740}$	$0.0026_{\pm 0.0005}$	$0.0109_{\pm 0.0003}$	$0.0004_{\pm 0.0001}$
1000	-	$0.0184_{\pm < 10^{-4}}$	$0.0879_{\pm 0.0102}$	$0.0017_{\pm 0.0004}$
2500	-	$0.2872_{\pm < 10^{-4}}$	$1.2480_{\pm 0.0075}$	$0.0107_{\pm 0.0030}$
5000	-	$2.2858_{\pm 0.0155}$	$18.0988_{\pm 0.1406}$	$0.0528_{\pm 0.0077}$
7500	-	$7.6892_{\pm 0.0248}$	$71.1197_{\pm 0.2178}$	$0.1380_{\pm 0.0345}$
10000	-	$18.2938_{\pm 0.0397}$	$165.0960_{\pm 1.2785}$	$0.2579_{\pm 0.0272}$

Table 1: **Execution times and standard deviations (mean $\pm$ SD) across implementation methods.** Values represent the average of 10 independent trials. For each matrix size N , the top-performing method is indicated in **bold**, while the second-best is underlined. Results for $C = 8$ CPU cores.

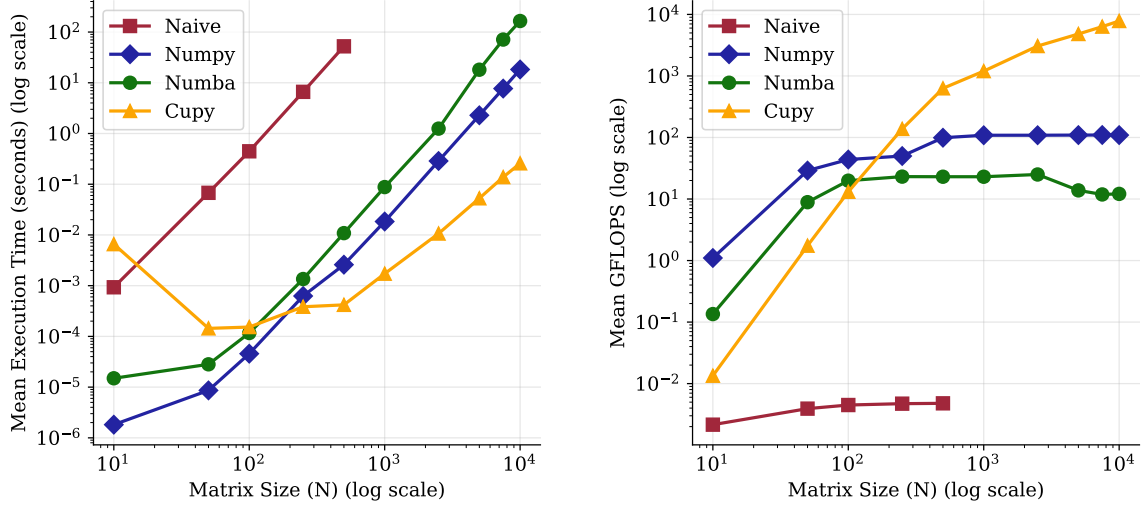


Figure 1: **Comparative performance analysis of matrix multiplication implementations.** (Left) Mean execution time as a function of matrix size N ; lower vertical positioning indicates superior temporal efficiency. (Right) Computational throughput in GFLOPS; higher values represent better hardware utilization. Both plots utilize logarithmic scales to visualize performance across several orders of magnitude. The Naive Python implementation is truncated at $N > 500$ due to the prohibitive execution time resulting from its $\mathcal{O}(N^3)$ complexity. Results for $C = 8$ CPU cores.

3. Experiments and results

The experimental framework involved the multiplication of randomly generated square matrices $M \in \mathbb{R}^{N \times N}$, where $N \in \{10, 50, 100, 250, 500, 1000, 2500, 5000, 7500, 10000\}$. Due to the cubic complexity of the algorithm, the naive implementation was limited to $N \leq 500$. To evaluate the impact of core count on computational performance, I benchmarked naive Python, NumPy, and Numba implementations across configurations of 8 and 16 CPU cores ($C = 8$ and $C = 16$). To ensure statistical significance and mitigate the impact of asynchronous execution overhead or JIT compilation latency, each N value was replicated across 10 independent trials. Each trial consisted of 5 internal iterations, with the mean execution time used for subsequent analysis.

Hardware specifications in this experiment are detailed in Appendix A.

Mean execution times across implementation methods are detailed in Table 1. The data indicate that for smaller matrices ($N < 250$), NumPy is the preferred method; however, for larger dimensions, the GPU-accelerated CuPy approach achieves superior performance. NumPy performed robustly across all test cases, consistently ranking as the best or second-best solution. Conversely, the naive approach exhibited severe performance degradation across all scales, as illustrated in Figure 1.

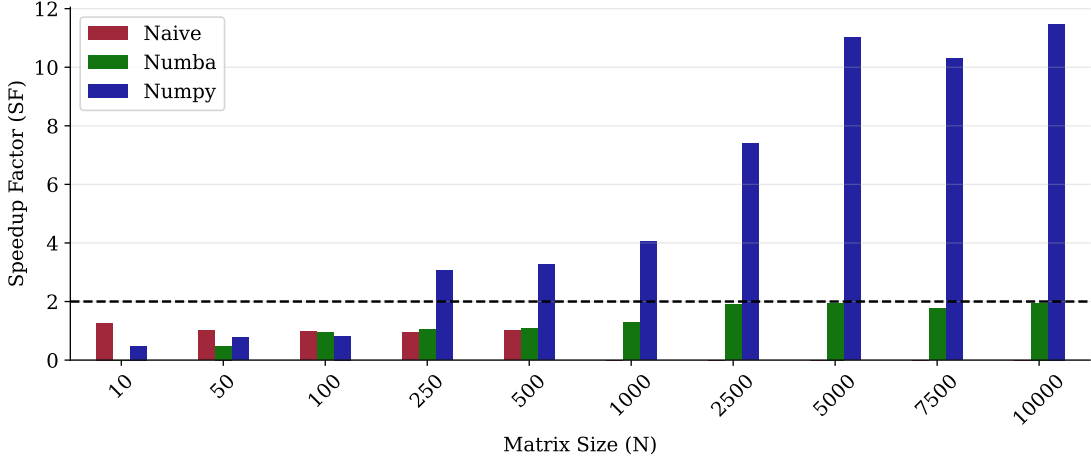


Figure 2: **Multi-core scaling efficiency across implementation methods.** The chart illustrates the Speedup Factor ($SF = T_{C=8}/T_{C=16}$) as a function of matrix size N . The dashed horizontal line at $SF = 2.0$ represents ideal linear scaling. Values below 1.0 at small N demonstrate performance degradation caused by parallelization overhead. For $N \geq 5000$, NumPy exhibits super-linear scaling ($SF > 11$), indicating that the 16-core configuration benefits from significantly reduced memory bottlenecks and increased cache availability compared to the 8-core baseline.

When plotted on a logarithmic scale (Figure 1, left subplot), the linear trajectory of the execution times confirms the theoretical $\mathcal{O}(N^3)$ complexity of the task, regardless of the selected method. For very small $N \in \{10, 50\}$, the CuPy execution curve arcs upward, slower times than both NumPy and Numba. This empirically shows the concept of asynchronous data overhead – the latency incurred by transferring data from the CPU to the GPU exceeds the compute time of the mathematical operations themselves.

The analysis of computational throughput further isolates the software bottleneck. The plateau around 0 associated with the naive approach (Figure 1, right subplot) demonstrates that despite the availability of a powerful CPU, the hardware remains idle, stalled by the temporal overhead of the Python interpreter. The steep ascent of the optimized curves illustrates proper hardware saturation via vectorization and parallelization as matrix sizes scale. The subsequent plateau dictates the maximum throughput for the hardware-software configuration: at $C = 16$ CPU cores, NumPy reaches an absolute ceiling of approximately 1200 GFLOPS, while Numba plateaus at 25 GFLOPS.

Evaluating the GFLOPS metric at $N = 500$ on 16 CPU cores validates the core thesis: powerful hardware necessitates efficient code. Under these conditions, the naive implementation yields a negligible 0.0047 GFLOPS. In contrast, NumPy outputs ~ 330 GFLOPS – a staggering 70,000 $\times$ improvement on the exact same hardware. The CPU has massive vectorization and caching capabilities, but the unoptimized $\mathcal{O}(N^3)$ loops remain fundamentally incapable of addressing them.

Size	NumPy	Numba	Size	NumPy	Numba
10	0.0002	0.0002	1000	0.0002	0.0211
50	0.0002	0.0002	2500	0.0002	0.0002
100	0.0002	0.0028	5000	0.0002	0.0002
250	0.0002	0.0028	7500	0.0002	0.0002
500	0.0002	0.0002	10000	0.0002	0.0002

Table 2: **Statistical significance of performance differences between 8 and 16 CPU cores.** The tables present two-sided Mann-Whitney U test p -values comparing execution times for NumPy and Numba implementations across various problem sizes. A significance level of $\alpha = 0.05$ was used, with significant differences ($p < 0.05$) highlighted in **bold**. The test confirms that modifying the core count yields a statistically significant difference in execution time across all tested configurations.

A subsequent analysis examined the Speedup Factor $SF = \frac{T_{C=8}}{T_{C=16}}$ to evaluate the parallel scaling. As depicted in Figure 2, Numba achieves near-ideal linear scaling for $N > 1000$. Remarkably, NumPy records an $11.5\times$ speedup at $N = 10000$. Because this exceeds the theoretical compute boundary of a $2\times$ speedup when doubling the cores count, it likely indicates a super-linear speedup. This suggests the 8-core configuration hit a severe memory bottleneck, whereas the 16-core configuration — likely benefiting from expanded cumulative L3 cache and distributed memory channels — handled the task efficiently. This shows that software efficiency must be designed with a deep awareness of hardware architecture.

Finally, to confirm that the performance gains from parallelization were statistically robust, the calculation times for the NumPy and Numba methods across 8 and 16 cores were subjected to hypothesis testing. The normality of the execution time distributions was first evaluated using the Shapiro-Wilk test. Because the assumption of normality was not met across all subsets, and to keep consistent statistical rigor, the non-parametric Mann-Whitney U test was applied. The procedure confirmed that in all observed cases, the differences were statistically significant (Table 2). This statistically proves that scaling hardware resources yields tangible performance gains only when paired with algorithms capable of parallel execution.

4. Conclusions and limitations

The findings of this study demonstrate that modern hardware provides a theoretical upper bound of performance and highly optimized code is crucial to realize this potential. While hardware scaling, specifically the transition to many-core architectures, offers significant opportunities for acceleration, these resources remain dormant when paired with naive implementations. The observed $70,000\times$ performance gap between naive Python loops and optimized NumPy libraries serves as a solid reminder that as datasets scale toward the petabyte level, algorithmic efficiency is a fundamental requirement for computational fea-

sibility. Furthermore, the evidence of super-linear scaling indicates that efficiency must be engineered with a deep awareness of the memory hierarchy and hardware architecture, not simply raw CPU core counts.

A notable limitation of this study is that matrix multiplication is among the most heavily optimized operations in computing, with BLAS libraries refined over decades. The performance gap observed between naive and optimized implementations may therefore be larger than what would be found in less-optimized problem domains, and future work should validate these findings across a broader range of algorithms.

Finally, in an era defined by the economic costs of cloud computing and the environmental necessity for energy-efficient AI, the development of powerful CPUs does not diminish the value of efficient code – it amplifies it.

Statement

I hereby declare that I have not used any text generating tools to write text or code for this essay.

References

- C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.
- J. L. Hennessy and D. A. Patterson. *Computer architecture: a quantitative approach*. Elsevier, 2011.
- M. S. Lam, E. E. Rothberg, and M. E. Wolf. The cache performance and optimizations of blocked algorithms. In *Proceedings of the fourth international conference on Architectural support for programming languages and operating systems*, pages 63–74, 1991.
- S. K. Lam, A. Pitrou, and S. Seibert. Numba: A llvm-based python jit compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, pages 1–6, 2015.
- G. E. Moore. Cramming more components onto integrated circuits. *Electronics*, 38(8):114–117, 1965.
- R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Crabb. Cupy: A numpy-compatible library for nvidia gpu calculations. In *Proceedings of Workshop on Machine Learning Systems (LearningSys) in The Thirty-first Annual Conference on Neural Information Processing Systems (NIPS)*, 2017.
- E. Strubell, A. Ganesh, and A. McCallum. Energy and policy considerations for deep learning in nlp. *arXiv preprint arXiv:1906.02243*, 2019.
- N. Wirth. A plea for lean software. *Computer*, 28(2):64–68, 1995.

Appendix

A. Hardware details

All computational experiments were executed on an NVIDIA DGX A100 system, with resource allocation and job scheduling orchestrated by the Slurm Workload Manager. The underlying hardware node is equipped with eight NVIDIA A100 Tensor Core GPUs linked via six NVIDIA NVSwitches, delivering 4.8 TB/s of bidirectional bandwidth and capable of up to 5 petaFLOPS of AI performance. Depending on the specific configuration utilized, the system provides either 320 GB or 640 GB of total GPU memory. The host architecture features dual AMD Rome 7742 processors, supplying a total of 128 CPU cores with a base clock speed of 2.25 GHz (up to 3.4 GHz max boost), supported by 1 TB to 2 TB of system memory. High-performance networking is facilitated by up to ten NVIDIA ConnectX-6 VPI adapters supporting 200 Gb/s HDR InfiniBand, while internal storage is handled by up to 30 TB of NVMe U.2 SSDs.

Note: Hardware specifications are adapted from the official technical documentation of the utilized HPC cluster.

B. Full results

Method	Size	Run ID	8 Cores		16 Cores	
			Time (s)	GFLOPS	Time (s)	GFLOPS
naive	10	0	0.0009	0.0021	0.0008	0.0024
naive	10	1	0.0009	0.0021	0.0008	0.0024
naive	10	2	0.0009	0.0022	0.0008	0.0024
naive	10	3	0.0009	0.0021	0.0007	0.0029
naive	10	4	0.0009	0.0022	0.0007	0.0029
naive	10	5	0.0009	0.0022	0.0007	0.0029
naive	10	6	0.0009	0.0021	0.0007	0.0029
naive	10	7	0.0009	0.0021	0.0007	0.0029
naive	10	8	0.0009	0.0022	0.0007	0.0029
naive	10	9	0.0009	0.0022	0.0007	0.0029
naive	50	0	0.1135	0.0022	0.0852	0.0029
naive	50	1	0.0960	0.0026	0.0847	0.0030
naive	50	2	0.0708	0.0035	0.0844	0.0030
naive	50	3	0.0574	0.0044	0.0710	0.0035
naive	50	4	0.0564	0.0044	0.0574	0.0044
naive	50	5	0.0566	0.0044	0.0565	0.0044
naive	50	6	0.0562	0.0044	0.0562	0.0044
naive	50	7	0.0565	0.0044	0.0563	0.0044
naive	50	8	0.0565	0.0044	0.0563	0.0044
naive	50	9	0.0574	0.0044	0.0562	0.0044
naive	100	0	0.4499	0.0044	0.4503	0.0044
naive	100	1	0.4463	0.0045	0.4506	0.0044

Method	Size	Run ID	8 Cores		16 Cores	
			Time (s)	GFLOPS	Time (s)	GFLOPS
naive	100	2	0.4454	0.0045	0.4500	0.0044
naive	100	3	0.4453	0.0045	0.4505	0.0044
naive	100	4	0.4449	0.0045	0.4500	0.0044
naive	100	5	0.4451	0.0045	0.4504	0.0044
naive	100	6	0.4442	0.0045	0.4501	0.0044
naive	100	7	0.4451	0.0045	0.4502	0.0044
naive	100	8	0.4448	0.0045	0.4501	0.0044
naive	100	9	0.4461	0.0045	0.4505	0.0044
naive	250	0	6.9586	0.0045	7.0209	0.0045
naive	250	1	6.9562	0.0045	6.9977	0.0045
naive	250	2	6.9635	0.0045	7.0058	0.0045
naive	250	3	6.7776	0.0046	7.0100	0.0045
naive	250	4	6.4360	0.0049	7.0127	0.0045
naive	250	5	6.4345	0.0049	7.0085	0.0045
naive	250	6	6.4344	0.0049	7.0120	0.0045
naive	250	7	6.4334	0.0049	6.9951	0.0045
naive	250	8	6.4336	0.0049	7.0308	0.0044
naive	250	9	6.4402	0.0049	7.0318	0.0044
naive	500	0	52.8503	0.0047	52.3373	0.0048
naive	500	1	51.9760	0.0048	52.3776	0.0048
naive	500	2	51.8425	0.0048	51.9397	0.0048
naive	500	3	52.0719	0.0048	51.8427	0.0048
naive	500	4	52.1472	0.0048	52.0375	0.0048
naive	500	5	52.3777	0.0048	52.3373	0.0048
naive	500	6	52.6368	0.0047	52.3776	0.0048
naive	500	7	52.1206	0.0048	51.9397	0.0048
naive	500	8	52.1136	0.0048	51.8427	0.0048
naive	500	9	52.9169	0.0047	52.0375	0.0048
numpy	10	0	0.0000	1.0422	0.0000	0.2295
numpy	10	1	0.0000	1.1069	0.0000	0.2437
numpy	10	2	0.0000	1.1203	0.0000	0.7140
numpy	10	3	0.0000	1.1255	0.0000	0.7725
numpy	10	4	0.0000	1.0537	0.0000	0.7482
numpy	10	5	0.0000	1.1231	0.0000	0.7511
numpy	10	6	0.0000	1.1244	0.0000	0.7810
numpy	10	7	0.0000	1.1241	0.0000	0.7953
numpy	10	8	0.0000	1.1202	0.0000	0.7877
numpy	10	9	0.0000	1.0650	0.0000	0.7516
numpy	50	0	0.0000	28.3788	0.0000	21.3859
numpy	50	1	0.0000	28.8493	0.0000	19.2958
numpy	50	2	0.0000	29.3377	0.0000	21.5822
numpy	50	3	0.0000	28.9302	0.0000	22.9856
numpy	50	4	0.0000	29.0444	0.0000	23.2944

Method	Size	Run ID	8 Cores		16 Cores	
			Time (s)	GFLOPS	Time (s)	GFLOPS
numpy	50	5	0.0000	29.1922	0.0000	23.1348
numpy	50	6	0.0000	28.8223	0.0000	23.3335
numpy	50	7	0.0000	29.1088	0.0000	23.2597
numpy	50	8	0.0000	29.1223	0.0000	23.5272
numpy	50	9	0.0000	29.1877	0.0000	23.4386
numpy	100	0	0.0000	43.8040	0.0001	32.3908
numpy	100	1	0.0000	43.9216	0.0001	35.9360
numpy	100	2	0.0000	43.6619	0.0001	38.5390
numpy	100	3	0.0000	43.8566	0.0001	38.2453
numpy	100	4	0.0000	43.8076	0.0001	37.6497
numpy	100	5	0.0000	43.7291	0.0001	36.6311
numpy	100	6	0.0000	43.9278	0.0001	36.7106
numpy	100	7	0.0000	43.8234	0.0001	35.2956
numpy	100	8	0.0000	43.8497	0.0001	36.8339
numpy	100	9	0.0000	44.0030	0.0001	34.0507
numpy	250	0	0.0006	49.3856	0.0003	108.2246
numpy	250	1	0.0006	49.5763	0.0002	156.9064
numpy	250	2	0.0006	49.6987	0.0002	156.9221
numpy	250	3	0.0006	49.5647	0.0002	164.7411
numpy	250	4	0.0006	49.7551	0.0002	158.1107
numpy	250	5	0.0006	49.6089	0.0002	156.9237
numpy	250	6	0.0006	49.7910	0.0002	168.7829
numpy	250	7	0.0006	49.8541	0.0002	157.4004
numpy	250	8	0.0006	50.6687	0.0002	155.1549
numpy	250	9	0.0006	51.8410	0.0002	168.4329
numpy	500	0	0.0040	62.9466	0.0010	248.1536
numpy	500	1	0.0030	82.4839	0.0008	294.3788
numpy	500	2	0.0025	101.5603	0.0008	321.8439
numpy	500	3	0.0023	106.4300	0.0008	330.9882
numpy	500	4	0.0024	105.8345	0.0008	319.4876
numpy	500	5	0.0024	106.3609	0.0007	333.5133
numpy	500	6	0.0023	106.4092	0.0008	322.4135
numpy	500	7	0.0024	106.3296	0.0008	332.6730
numpy	500	8	0.0024	106.3030	0.0008	320.0400
numpy	500	9	0.0024	106.2691	0.0007	334.4763
numpy	1000	0	0.0184	108.7580	0.0052	385.8984
numpy	1000	1	0.0184	108.9600	0.0046	432.1950
numpy	1000	2	0.0184	108.9201	0.0044	456.2475
numpy	1000	3	0.0184	108.7954	0.0046	439.2983
numpy	1000	4	0.0184	108.5020	0.0044	458.2776
numpy	1000	5	0.0185	108.2872	0.0046	430.4363
numpy	1000	6	0.0184	108.7181	0.0044	458.7053
numpy	1000	7	0.0184	108.7405	0.0045	440.3247

Method	Size	Run ID	8 Cores		16 Cores	
			Time (s)	GFLOPS	Time (s)	GFLOPS
numpy	1000	8	0.0184	108.7291	0.0043	462.5814
numpy	1000	9	0.0184	108.6827	0.0045	441.6519
numpy	2500	0	0.2870	108.8768	0.0500	625.5069
numpy	2500	1	0.2870	108.8714	0.0466	669.9381
numpy	2500	2	0.2872	108.8089	0.0494	632.5773
numpy	2500	3	0.2872	108.8066	0.0463	674.6669
numpy	2500	4	0.2872	108.8072	0.0428	730.6054
numpy	2500	5	0.2871	108.8360	0.0387	808.3510
numpy	2500	6	0.2872	108.8058	0.0317	986.1715
numpy	2500	7	0.2872	108.7950	0.0280	1117.3393
numpy	2500	8	0.2872	108.8082	0.0275	1134.6227
numpy	2500	9	0.2871	108.8497	0.0280	1116.6740
numpy	5000	0	2.2714	110.0632	0.1949	1282.8407
numpy	5000	1	2.2879	109.2701	0.2064	1211.5196
numpy	5000	2	2.2949	108.9385	0.2082	1200.9908
numpy	5000	3	2.2912	109.1146	0.2055	1216.7228
numpy	5000	4	2.2735	109.9609	0.2097	1192.2827
numpy	5000	5	2.2735	109.9627	0.2104	1187.9667
numpy	5000	6	2.2725	110.0089	0.2097	1191.9846
numpy	5000	7	2.2725	110.0114	0.2092	1194.7899
numpy	5000	8	2.3075	108.3418	0.2097	1192.2722
numpy	5000	9	2.3128	108.0963	0.2117	1181.1185
numpy	7500	0	7.7238	109.2407	0.7458	1131.3202
numpy	7500	1	7.6899	109.7225	0.7482	1127.6400
numpy	7500	2	7.6813	109.8448	0.7475	1128.7987
numpy	7500	3	7.6896	109.7259	0.7424	1136.5895
numpy	7500	4	7.6697	110.0112	0.7502	1124.6688
numpy	7500	5	7.6591	110.1634	0.7495	1125.6998
numpy	7500	6	7.6981	109.6054	0.7501	1124.7765
numpy	7500	7	7.6700	110.0070	0.7495	1125.7001
numpy	7500	8	7.6736	109.9550	0.7403	1139.6806
numpy	7500	9	7.7370	109.0543	0.7424	1136.4802
numpy	10000	0	18.1992	109.8948	1.5856	1261.3446
numpy	10000	1	18.2928	109.3329	1.5925	1255.8638
numpy	10000	2	18.3253	109.1385	1.5899	1257.9195
numpy	10000	3	18.3107	109.2259	1.5965	1252.7392
numpy	10000	4	18.3055	109.2566	1.5983	1251.3478
numpy	10000	5	18.3488	108.9991	1.5951	1253.8485
numpy	10000	6	18.3017	109.2796	1.5957	1253.3662
numpy	10000	7	18.2685	109.4781	1.5968	1252.4800
numpy	10000	8	18.2867	109.3689	1.5958	1253.3247
numpy	10000	9	18.2983	109.2998	1.5951	1253.8492
numba	10	0	0.0000	0.1136	0.0181	0.0001

Method	Size	Run ID	8 Cores		16 Cores	
			Time (s)	GFLOPS	Time (s)	GFLOPS
numba	10	1	0.0000	0.1118	0.0182	0.0001
numba	10	2	0.0000	0.1338	0.0176	0.0001
numba	10	3	0.0000	0.1409	0.0096	0.0002
numba	10	4	0.0000	0.1412	0.0000	0.0509
numba	10	5	0.0000	0.1403	0.0000	0.0526
numba	10	6	0.0000	0.1422	0.0000	0.0543
numba	10	7	0.0000	0.1419	0.0000	0.0474
numba	10	8	0.0000	0.1416	0.0000	0.0525
numba	10	9	0.0000	0.1420	0.0000	0.0535
numba	50	0	0.0000	9.1637	0.0001	3.8226
numba	50	1	0.0000	8.3729	0.0001	4.3339
numba	50	2	0.0000	8.7666	0.0001	4.3045
numba	50	3	0.0000	8.5909	0.0001	4.1725
numba	50	4	0.0000	9.0051	0.0001	4.3494
numba	50	5	0.0000	8.8750	0.0001	4.3321
numba	50	6	0.0000	9.1463	0.0001	4.4231
numba	50	7	0.0000	9.0778	0.0001	4.4481
numba	50	8	0.0000	8.6588	0.0001	4.2384
numba	50	9	0.0000	9.1948	0.0001	4.4096
numba	100	0	0.0003	6.1737	0.0001	14.9565
numba	100	1	0.0001	21.2226	0.0001	14.9963
numba	100	2	0.0001	21.5885	0.0001	16.5749
numba	100	3	0.0001	21.1221	0.0001	16.4568
numba	100	4	0.0001	21.3922	0.0001	16.7741
numba	100	5	0.0001	21.0641	0.0001	17.0982
numba	100	6	0.0001	21.6128	0.0001	16.9128
numba	100	7	0.0001	21.6465	0.0001	16.7310
numba	100	8	0.0001	21.2638	0.0001	16.8415
numba	100	9	0.0001	21.6011	0.0001	16.4729
numba	250	0	0.0014	22.3725	0.0015	21.1115
numba	250	1	0.0014	22.7382	0.0012	25.1669
numba	250	2	0.0014	23.1386	0.0013	24.5585
numba	250	3	0.0013	23.3776	0.0012	25.0765
numba	250	4	0.0013	23.1724	0.0013	24.2446
numba	250	5	0.0013	23.3488	0.0012	25.0292
numba	250	6	0.0013	23.1509	0.0013	24.3549
numba	250	7	0.0013	23.4247	0.0013	24.8615
numba	250	8	0.0014	23.1423	0.0013	24.6169
numba	250	9	0.0013	23.2647	0.0012	25.1897
numba	500	0	0.0108	23.1245	0.0102	24.5416
numba	500	1	0.0108	23.2423	0.0100	25.0663
numba	500	2	0.0107	23.3190	0.0101	24.7519
numba	500	3	0.0107	23.4631	0.0107	23.4076

Method	Size	Run ID	8 Cores		16 Cores	
			Time (s)	GFLOPS	Time (s)	GFLOPS
numba	500	4	0.0108	23.2554	0.0105	23.7991
numba	500	5	0.0107	23.3391	0.0102	24.4446
numba	500	6	0.0108	23.1905	0.0101	24.7839
numba	500	7	0.0107	23.3226	0.0100	24.9578
numba	500	8	0.0117	21.4040	0.0097	25.7963
numba	500	9	0.0111	22.6025	0.0094	26.7254
numba	1000	0	0.1062	18.8274	0.0896	22.3123
numba	1000	1	0.0982	20.3734	0.0962	20.7984
numba	1000	2	0.0980	20.4064	0.0927	21.5726
numba	1000	3	0.0944	21.1803	0.0772	25.8994
numba	1000	4	0.0798	25.0714	0.0642	31.1512
numba	1000	5	0.0800	24.9927	0.0504	39.6612
numba	1000	6	0.0806	24.8041	0.0604	33.0985
numba	1000	7	0.0808	24.7439	0.0567	35.2796
numba	1000	8	0.0804	24.8786	0.0504	39.6603
numba	1000	9	0.0804	24.8772	0.0461	43.3530
numba	2500	0	1.2576	24.8481	0.6519	47.9365
numba	2500	1	1.2548	24.9045	0.6654	46.9626
numba	2500	2	1.2479	25.0413	0.6454	48.4196
numba	2500	3	1.2481	25.0380	0.6680	46.7786
numba	2500	4	1.2355	25.2928	0.6477	48.2503
numba	2500	5	1.2497	25.0069	0.6463	48.3516
numba	2500	6	1.2534	24.9326	0.6493	48.1264
numba	2500	7	1.2498	25.0037	0.6492	48.1343
numba	2500	8	1.2481	25.0376	0.6490	48.1478
numba	2500	9	1.2348	25.3076	0.6580	47.4899
numba	5000	0	18.3920	13.5928	9.3160	26.8356
numba	5000	1	18.1028	13.8100	9.3382	26.7716
numba	5000	2	17.9047	13.9628	9.1892	27.2058
numba	5000	3	18.2763	13.6789	9.2141	27.1323
numba	5000	4	18.0809	13.8268	9.4446	26.4701
numba	5000	5	18.1035	13.8095	9.2072	27.1526
numba	5000	6	17.9742	13.9088	9.2794	26.9414
numba	5000	7	18.0462	13.8534	9.1894	27.2053
numba	5000	8	18.0546	13.8469	9.2883	26.9155
numba	5000	9	18.0523	13.8486	9.2336	27.0750
numba	7500	0	71.1749	11.8546	40.1801	20.9992
numba	7500	1	71.2507	11.8420	40.6175	20.7731
numba	7500	2	70.9099	11.8989	40.2102	20.9835
numba	7500	3	71.4045	11.8165	40.0633	21.0604
numba	7500	4	70.8587	11.9075	40.1613	21.0090
numba	7500	5	71.1749	11.8546	40.6007	20.7817
numba	7500	6	71.2507	11.8420	39.8534	21.1714

Method	Size	Run ID	8 Cores		16 Cores	
			Time (s)	GFLOPS	Time (s)	GFLOPS
numba	7500	7	70.9099	11.8989	39.5352	21.3417
numba	7500	8	71.4045	11.8165	40.1894	20.9943
numba	7500	9	70.8587	11.9075	41.0809	20.5387
numba	10000	0	162.7663	12.2876	85.7064	23.3355
numba	10000	1	166.1391	12.0381	84.6818	23.6178
numba	10000	2	165.4454	12.0886	85.9709	23.2637
numba	10000	3	165.1910	12.1072	85.8088	23.3076
numba	10000	4	165.9383	12.0527	84.6447	23.6282
numba	10000	5	162.7663	12.2876	86.8851	23.0189
numba	10000	6	166.1391	12.0381	84.9670	23.5386
numba	10000	7	165.4454	12.0886	83.9277	23.8300
numba	10000	8	165.1910	12.1072	83.8485	23.8525
numba	10000	9	165.9383	12.0527	79.9403	25.0187
cupy	10	0	0.0647	0.0000	-	-
cupy	10	1	0.0001	0.0144	-	-
cupy	10	2	0.0001	0.0146	-	-
cupy	10	3	0.0001	0.0150	-	-
cupy	10	4	0.0001	0.0148	-	-
cupy	10	5	0.0001	0.0147	-	-
cupy	10	6	0.0001	0.0152	-	-
cupy	10	7	0.0001	0.0149	-	-
cupy	10	8	0.0001	0.0152	-	-
cupy	10	9	0.0001	0.0153	-	-
cupy	50	0	0.0002	1.4541	-	-
cupy	50	1	0.0001	1.7465	-	-
cupy	50	2	0.0001	1.7757	-	-
cupy	50	3	0.0001	1.7931	-	-
cupy	50	4	0.0001	1.7983	-	-
cupy	50	5	0.0001	1.7856	-	-
cupy	50	6	0.0001	1.7712	-	-
cupy	50	7	0.0001	1.8134	-	-
cupy	50	8	0.0001	1.7264	-	-
cupy	50	9	0.0001	1.7675	-	-
cupy	100	0	0.0002	12.0469	-	-
cupy	100	1	0.0002	13.2437	-	-
cupy	100	2	0.0002	12.4999	-	-
cupy	100	3	0.0002	13.0702	-	-
cupy	100	4	0.0002	13.2544	-	-
cupy	100	5	0.0002	13.2006	-	-
cupy	100	6	0.0001	13.4856	-	-
cupy	100	7	0.0001	13.4340	-	-
cupy	100	8	0.0001	13.4449	-	-
cupy	100	9	0.0001	13.3528	-	-

Method	Size	Run ID	8 Cores		16 Cores	
			Time (s)	GFLOPS	Time (s)	GFLOPS
cupy	250	0	0.0020	15.8149	-	-
cupy	250	1	0.0002	149.3945	-	-
cupy	250	2	0.0002	151.5651	-	-
cupy	250	3	0.0002	151.8635	-	-
cupy	250	4	0.0002	154.5067	-	-
cupy	250	5	0.0002	152.5884	-	-
cupy	250	6	0.0002	152.6989	-	-
cupy	250	7	0.0002	154.7629	-	-
cupy	250	8	0.0002	133.0011	-	-
cupy	250	9	0.0002	161.4068	-	-
cupy	500	0	0.0008	313.8392	-	-
cupy	500	1	0.0004	649.9118	-	-
cupy	500	2	0.0004	664.8561	-	-
cupy	500	3	0.0004	673.4334	-	-
cupy	500	4	0.0004	665.4805	-	-
cupy	500	5	0.0004	662.1709	-	-
cupy	500	6	0.0004	669.4727	-	-
cupy	500	7	0.0004	663.4139	-	-
cupy	500	8	0.0004	660.9783	-	-
cupy	500	9	0.0004	666.5508	-	-
cupy	1000	0	0.0028	710.9255	-	-
cupy	1000	1	0.0016	1274.5026	-	-
cupy	1000	2	0.0016	1276.5288	-	-
cupy	1000	3	0.0016	1266.9171	-	-
cupy	1000	4	0.0016	1264.0963	-	-
cupy	1000	5	0.0018	1103.3948	-	-
cupy	1000	6	0.0016	1266.9541	-	-
cupy	1000	7	0.0016	1275.4570	-	-
cupy	1000	8	0.0016	1271.9510	-	-
cupy	1000	9	0.0016	1263.0356	-	-
cupy	2500	0	0.0191	1637.0936	-	-
cupy	2500	1	0.0100	3127.2317	-	-
cupy	2500	2	0.0094	3312.7320	-	-
cupy	2500	3	0.0100	3128.1292	-	-
cupy	2500	4	0.0094	3323.4731	-	-
cupy	2500	5	0.0100	3140.5262	-	-
cupy	2500	6	0.0094	3334.5081	-	-
cupy	2500	7	0.0099	3151.7576	-	-
cupy	2500	8	0.0094	3321.3760	-	-
cupy	2500	9	0.0100	3134.7004	-	-
cupy	5000	0	0.0747	3348.1567	-	-
cupy	5000	1	0.0505	4952.8491	-	-
cupy	5000	2	0.0503	4968.7532	-	-

Method	Size	Run ID	8 Cores		16 Cores	
			Time (s)	GFLOPS	Time (s)	GFLOPS
cupy	5000	3	0.0504	4964.5516	-	-
cupy	5000	4	0.0503	4965.4453	-	-
cupy	5000	5	0.0503	4967.3386	-	-
cupy	5000	6	0.0502	4975.2739	-	-
cupy	5000	7	0.0503	4971.1852	-	-
cupy	5000	8	0.0503	4970.9268	-	-
cupy	5000	9	0.0502	4975.4221	-	-
cupy	7500	0	0.2363	3570.9705	-	-
cupy	7500	1	0.1271	6636.2997	-	-
cupy	7500	2	0.1270	6645.4559	-	-
cupy	7500	3	0.1273	6626.3054	-	-
cupy	7500	4	0.1269	6647.4184	-	-
cupy	7500	5	0.1274	6624.4380	-	-
cupy	7500	6	0.1269	6646.4991	-	-
cupy	7500	7	0.1273	6626.6919	-	-
cupy	7500	8	0.1267	6661.3633	-	-
cupy	7500	9	0.1272	6632.7247	-	-
cupy	10000	0	0.3354	5963.7142	-	-
cupy	10000	1	0.2495	8017.3402	-	-
cupy	10000	2	0.2491	8028.4130	-	-
cupy	10000	3	0.2499	8003.8285	-	-
cupy	10000	4	0.2491	8029.7864	-	-
cupy	10000	5	0.2492	8024.3244	-	-
cupy	10000	6	0.2498	8005.2629	-	-
cupy	10000	7	0.2488	8037.1206	-	-
cupy	10000	8	0.2488	8037.8948	-	-
cupy	10000	9	0.2495	8015.5906	-	-